Trabajo final de Tópicos II - 2026

Enunciado

El cliente necesita una plataforma de ticketera virtual para eventos. El sistema debe gestionar tres tipos de usuarios: administradores (gestión completa, mantenimiento y tareas operativas), event owners (diseño y publicación de eventos) y clientes (compra de tickets para eventos listados).

La plataforma debe enfrentar los siguientes desafíos:

A continuación se clasifican los desafíos de la plataforma en funcionales (relacionados con el comportamiento y las capacidades del negocio) y técnicos (relacionados con la arquitectura, el rendimiento y la infraestructura).

Desafíos funcionales

Aquellos que definen qué debe hacer el sistema desde la perspectiva del negocio y del usuario.

Desafío	Descripción
Garantizar que un ticket no se venda dos veces	Asegurar unicidad de venta por ticket, aplicando reglas de negocio de idempotencia y control de concurrencia a nivel funcional.
Gestionar devoluciones	Permitir que clientes soliciten devoluciones y que administradores las aprueben o rechacen, liberando el asiento/zona correspondiente.

Soportar eventos masivos (colas de venta)	Definir el comportamiento del sistema ante alta demanda: encolar solicitudes, notificar posición y procesar en orden.
Entradas numeradas y no numeradas	Soportar dos modalidades de ticket: con asiento específico (numerado) y por zona (no numerado), con reglas de negocio distintas para cada una.
Sistema de recomendación (Etapa II)	Generar recomendaciones personalizadas de eventos según historial y preferencias del cliente.
Búsquedas por múltiples criterios	Permitir búsqueda de eventos por tipo, organizador, participante y fecha, con ordenamiento por relevancia.

Desafíos técnicos

Aquellos que definen cómo debe construirse y operarse el sistema para cumplir los desafíos funcionales de forma robusta, escalable y mantenible.

Desafío	Descripción
Garantizar que un ticket no se venda dos veces (idempotencia y control de concurrencia)	Implementar mecanismos técnicos como locks optimistas/pesimistas, transacciones distribuidas, claves de idempotencia y de duplicación para evitar doble venta.

Arquitectura de microservicios	Definir límites de servicios, comunicación entre ellos (sync/async), manejo de fallos parciales, consistencia eventual y despliegue independiente.
Bitácora de solicitudes HTTP con medición de tiempos	Instrumentar cada solicitud HTTP: iniciar timer al recibirla, procesar autenticación, registrar resultado y tiempo en la bitácora, y retornar la respuesta.
Estrategias de cache	Definir qué cachear, dónde (cliente, CDN, servicio, distribuido), política de invalidación y TTL según el tipo de dato.
Soportar eventos masivos (colas de venta)	Implementar infraestructura de colas, backpressure, procesamiento asíncrono y notificaciones en tiempo real.

Metodología

Desarrollo guiado por especificaciones con OpenSpec, en tres etapas incrementales:

1. Etapa I — Core: backend, APIs, autenticación, compra de tickets, mockups
2. Etapa II — Integración con sistema de recomendación y cache
3. Etapa III — Escalabilidad: colas de venta, devoluciones, búsqueda avanzada y stress test

Cada etapa se gestiona como un `change` en OpenSpec (`proposal → specs → design → tasks → implement → archive`).

Entregables

- `CONTRIBUTE.md` con flujo de trabajo (feature branch, tests, CI)
- Changes archivados con specs, design y tasks
- Mockups, APIs funcionales y tests de unidad
- Bitácora de documentación